

**Argus**

Built by ZTPL

Product Specification

A complete technical and functional overview of Argus — architecture, monitoring engine, alerting, integrations, security, and deployment.

CONTENTS

Overview	Architecture & stack	Core monitoring engine
Discovery & classification	Alerting & escalation	Notification channels
Firewall & vendor integrations	SNMP support	Remote agents (multi-site / segmented networks)
Security	Licensing	Deployment & operations
Dashboard & UI	System requirements	

Overview

Argus is a self-contained network monitoring system that watches every device on a network — routers, switches, servers, firewalls, cameras, access points — and tells the right person the moment one goes down. It ships as a single Windows executable with no external dependencies: no database server to install, no account to create, no vendor cloud in the path between a device going down and someone finding out.

Everything — device inventory, historical metrics, alert history, configuration — lives in one local SQLite file next to the install. Nothing leaves the network unless explicitly configured to (an outbound webhook, an SMTP relay, a Slack webhook).

Architecture & stack

Backend: Bun + TypeScript, built on a hexagonal (ports-and-adapters) architecture — domain logic has zero dependency on any specific database, notification channel, or network library, which is what lets the same monitoring engine run identically whether backed by SQLite (the shipped Windows product) or Postgres (the multi-tenant SaaS deployment). Frontend: React + Vite + Tailwind, with Recharts/ECharts for data visualization.

Runtime	Bun (single compiled executable, no Node.js/npm required on the target machine)
Database	SQLite (embedded, exe mode) or PostgreSQL (multi-tenant SaaS mode) — same domain logic, swappable adapter
Backend framework	Hono (lightweight HTTP layer)
Frontend	React 18, Vite, Tailwind CSS, Recharts/ECharts, Framer Motion
Architecture	Hexagonal / ports-adapters — src/{domain, application, adapters, api, bootstrap}
Distribution	Single compiled .exe (portable) + an optional Inno Setup wizard installer

Core monitoring engine

Every device is monitored through one or more independent checks, each polled on its own interval:

- ICMP — ping-based reachability, latency, and packet loss.
- TCP — port-level reachability for any service.
- HTTP/HTTPS — status code, response body match, TLS validation (with a self-signed override for lab/internal appliances).
- SNMP v1/v2c/v3 — CPU, memory, interface throughput/status, and general device identity (sysDescr/sysName/sysObjectId).
- Vendor REST/XML APIs — FortiGate and Sophos, see Firewall Integrations below.

A deterministic state machine (up → degraded → down → flapping, plus a maintenance override) decides device status from check outcomes — configurable

consecutive-failure/consecutive-recovery thresholds avoid flapping on a single dropped ping, and a flap-detector automatically suppresses noisy alerting on a device that's rapidly bouncing between states instead of paging on every single transition.

ICMP is authoritative for reachability when configured: a secondary check (SNMP, HTTP, a vendor API) failing — a stale credential, a firewall rule, a misconfiguration — is recorded and visible on that check specifically, but does not by itself mark a genuinely reachable device as down. This deliberately avoids false "it's down" pages caused by a monitoring-configuration problem rather than an actual outage.

Polling is tiered: devices in a degraded or uncertain state are polled more frequently than a stable, healthy device, so a real problem is caught fast without polling a quiet, healthy fleet harder than it needs.

Maintenance windows (one-off or recurring, scoped to a single device or an entire group) suppress alerting for planned work without disabling monitoring itself — the real state keeps being tracked underneath, so alerting resumes instantly and accurately the moment the window ends.

Discovery & classification

Point Argus at a CIDR range and it sweeps the subnet via ARP + ICMP + TCP port probing + SNMP, then classifies each responding device (router, switch, server, camera, firewall, access point, printer, NAS, IoT, workstation, or unknown) using a weighted signal model: open-port fingerprint, SNMP sysDescr string, MAC-address vendor (OUI) lookup, and DHCP/reverse-DNS hostname. Each guess ships with a confidence score, shown as a color-coded badge so an admin can tell at a glance which imports are safe to accept as-is and which are worth a manual check.

Classification isn't a one-time guess: devices still sitting at type "unknown" are automatically re-probed on a daily schedule and reclassified once a strong signal (SNMP sysDescr) becomes available — without ever overriding a type an admin has already set manually.

Alerting & escalation

Alerts carry a severity (info / warning / critical) and a lifecycle (open → acknowledged → resolved), with a full per-alert timeline and free-text investigation notes. Tiered escalation notifies the device's owner first, then works up a configurable per-group chain (e.g. team lead at +10 minutes, on-call admin at +30) if nobody acknowledges — and stops the instant someone does.

- Critical-asset instant paging — cameras and firewalls (or anything else flagged) page immediately on DOWN, skipping the storm-buffer delay and hourly rate limit every other device gets, since their downtime is always actionable regardless of what else is happening on the network.
- Storm guard — a genuinely correlated outage (e.g. a whole branch's devices dropping at once because their shared uplink died) is aggregated into a single storm notification instead of flooding every recipient with one alert per affected device.
- Quiet hours + severity floor — each person sets their own minimum severity and do-not-disturb window; critical alerts always break through regardless.
- On-call rotations and per-group escalation chains, configurable independently of the default owner-first flow.
- One-click email acknowledgement links — an HMAC-signed URL that acknowledges an alert straight from an email client, no login required.
- Opt-in personal digest emails (daily or weekly SLA + open-alerts summary), independent of real-time alerting.

Notification channels

Each person configures their own delivery targets independently, on top of the instance-wide SMTP/webhook setup an admin manages:

- Email — via the instance's own SMTP relay.
- Generic webhook — signed with an HMAC secret so the receiving endpoint can verify authenticity.
- Slack — posts a severity-colored message to an Incoming Webhook.
- Microsoft Teams — posts a MessageCard to a channel's Incoming Webhook connector, with a one-click Acknowledge action.
- PagerDuty — full Events API v2 integration: triggers and resolves a real PagerDuty incident (not just a notification) as the underlying alert opens and resolves, correlated by a stable dedup key.
- Syslog (CEF format) — for forwarding into an existing SIEM.

Firewall & vendor integrations

A dedicated Firewalls dashboard gives a fleet-wide, at-a-glance view of every firewall device — live CPU/memory/session/ VPN-tunnel metrics alongside model, firmware version, and HA role.

FortiGate

A real REST API adapter against FortiGate's monitor API (`/api/v2/monitor/*`) — CPU, memory, active sessions, IPsec VPN tunnel status, plus identity facts (model, firmware build, serial number, HA role) pulled straight from the device, not admin-entered.

Sophos

A REST/XML adapter against standalone on-prem Sophos Firewall's (SFOS/XG) classic management API — authenticates and validates reachability, and reports IPsec VPN tunnel status. Sophos's on-prem API has no live CPU/memory telemetry endpoint the way FortiGate's does; pair a Sophos device with SNMP (below) for those metrics.

Any other firewall vendor (SonicWall, WatchGuard, pfSense, Check Point, Palo Alto, and more) is fully supported through the generic SNMP checker — classification, CPU/memory, and interface bandwidth all work identically regardless of brand.

SNMP support

Full SNMP v1, v2c, and v3 support (auth+priv, all standard protocols) for any SNMP-capable device — not just firewalls.

- Bandwidth & interface monitoring — per-interface in/out throughput, packet counters, and up/down status via the standard IF-MIB, with a live interface-discovery picker (walks the device's own ifTable) instead of requiring an admin to hand-snmpwalk a device to find index numbers.
- CPU & memory — via the standard HOST-RESOURCES-MIB, working identically across virtually any network-attached hardware.
- Device identity — sysDescr/sysName/sysObjectId, feeding both classification and the Firewalls dashboard's model/firmware display.

Remote agents (multi-site / segmented networks)

A single Argus instance can only directly reach devices on networks it can route to. For a genuinely segmented environment — separate VLANs, sites, or subnets with no direct route back to wherever the central instance runs — a lightweight standalone remote agent process runs inside that segment, performs ICMP/TCP/HTTP checks locally, and pushes results back to the central instance over an authenticated HTTPS connection.

Agents authenticate with their own purpose-built credential type (distinct from the read-only API keys used for external integrations), scoped device-by-device — an agent can only ever report results for devices an admin has explicitly assigned to it. Results flow through the exact same state-machine, alerting, and metrics pipeline a locally-pollled device uses, so an agent-monitored device behaves identically to a local one everywhere except how its raw check data was collected.

Security

- Two-factor authentication (TOTP, RFC 6238) with QR-code enrollment.
- Role-based access control — admin / operator / viewer, with optional per-user scoping to specific device groups.
- Session management — every active session visible and individually revocable from the account's own security page.
- SSO/SAML 2.0 (hosted/SaaS deployments) with auto-provisioning on first sign-in.
- Scoped, read-only API keys for external integrations (Prometheus scraping, CMDB sync, reporting) — enforced GET-only at the route level, not just by convention.
- Full audit log — every administrative action (device changes, user role changes, license application, settings changes) attributed to a user and timestamped, with a readable per-action description.
- Encrypted credential storage — SNMP community strings/v3 keys and vendor API credentials are encrypted at rest using the instance's own key, never stored or logged in plaintext.
- SSRF guards on every outbound-URL-accepting feature (webhooks, update checks, heartbeat pings) — link-local/metadata addresses are always blocked; private RFC1918 ranges are blocked specifically for notification targets (which should only ever point outward) while remaining fully allowed for device checks (which are the LAN, by design).

Licensing

Argus ships with a trial mode capped at a fixed device count, unlocked by a signed license file tied to a device-count plan. License state (valid / grace period / expired / invalid) is checked continuously, with a grace period after expiry before any functionality is reduced — monitoring never silently stops working the moment a renewal is late.

Deployment & operations

- Runs as a native Windows service (auto-start, survives reboot/logout) or as a portable standalone executable.
- Self-heartbeat dead-man's-switch — pings an external push-monitor (healthchecks.io, Cronitor, Uptime Kuma, or similar) so the one failure mode nothing inside the process can detect (the whole machine or service going down) still gets caught by something watching from outside.
- One-click backup/export and restore, plus automatic scheduled backups (daily/weekly, configurable retention).
- Self-update — checks a configurable update-feed URL, downloads and cryptographically verifies (SHA-256 + signature) a new build, and swaps itself in place on next restart.
- Sub-3-second cold start; a full release build is smoke-tested end-to-end (spawn → health check → setup → create a device → verify it persisted) before ever shipping.

Dashboard & UI

- Executive Overview dashboard — fleet-wide health, availability trend, latency leaders, and group-vs-group comparison at a glance.
- Inventory — sortable, filterable, taggable device list with bulk actions and CSV import/export.
- Live topology map — force-directed network graph, automatically centered, with critical-asset markers and rich per-node detail.
- Bandwidth — per-interface throughput charts across every SNMP-monitored device, with the live interface-discovery picker described above.
- SLA & reports — downtime/availability reporting per device, exportable and schedulable as an email summary.
- Full dark-mode support, keyboard command palette, and a kiosk mode for a wall-mounted NOC display.

System requirements

OS	Windows 10 / 11, 64-bit (Windows Server also supported for service deployment)
CPU / RAM	Any modern x64 CPU; 2 GB RAM minimum (scales with fleet size)
Disk	~150 MB for the application; metrics history scales with fleet size and retention settings
Network	Outbound access for SMTP/webhooks/update checks if used — otherwise fully offline-capable
Dependencies	None — single self-contained executable, no separate runtime or database install required

Ready to see it running on your own network?

[Download for Windows](#)